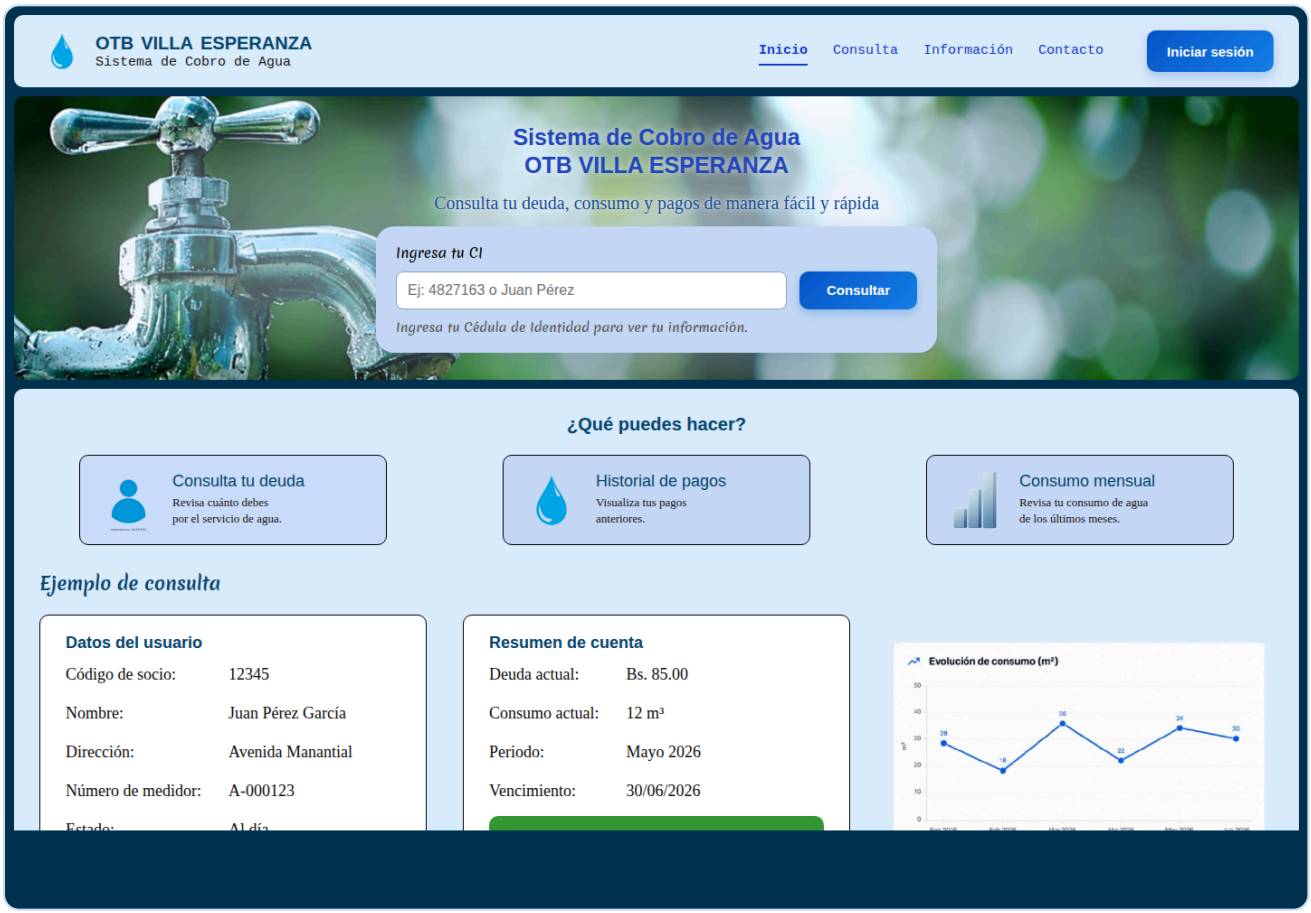
Sistema de Cobro de Agua

OTB Villa Esperanza

Qué cambié en el proyecto, cómo funciona y cómo está hecho el código

HTML CSS JavaScript



Página de inicio vista en computadora

Septiembre 2026

1 De qué trata el proyecto

Es una página web para la OTB Villa Esperanza. Sirve para que los vecinos vean cuánto deben del agua y puedan pagar con QR desde el celular. También tiene un panel para los cobradores y administradores, donde revisan y confirman los pagos.

Cuando recibí el proyecto solo estaba el diseño. Las páginas se veían bien en una computadora, pero los botones no hacían nada, los datos eran fijos (siempre salía Juan Pérez) y en el celular todo se salía de la pantalla. Lo que hice fue:

Responsive
La página se acomoda a computadora, tablet y celular.

JavaScript
Buscar, consultar la deuda, pagar con QR y mandar mensajes ya funciona.

Panel con usuarios
El personal de la OTB entra con usuario y contraseña.

Para abrirlo no hay que instalar nada. Se abre `index.html` con doble clic y carga en el navegador (Chrome, Firefox o Edge).

2 Qué cambié

2.1 Ordené las carpetas

Al principio todo estaba en una sola carpeta: páginas, estilos e imágenes mezclados. Separé cada tipo de archivo en su propia carpeta para que sea más fácil encontrar las cosas.

Antes

CARPETA

```
otb.html      otb1.css
index1.html   index1.css
index2.html   index2.css
index3.html   index3.css
agua-removebg-preview.png
grifo.jpeg
... (11 imágenes sueltas)
```

Ahora

CARPETA

```
index.html   → abre otb.html
otb.html     → Inicio
index1.html  → Consulta y pago QR
index2.html  → Información
index3.html  → Contacto
panel.html   → Panel privado
css/  estilos.css  sistema.css  panel.css
js/   datos.js  comun.js  publico.js
panel.js
img/  todas las imágenes + qr.JPG
```

2.2 Junté los 4 archivos CSS en uno

En el diseño original cada elemento se movía a mano con píxeles. Por ejemplo, el enlace *Inicio* estaba así:

```css
.inicio{
    position: relative;
    left: 1070px;     /* 1070 píxeles a la derecha */
    bottom: 245px;    /* 245 píxeles hacia arriba */
}
```

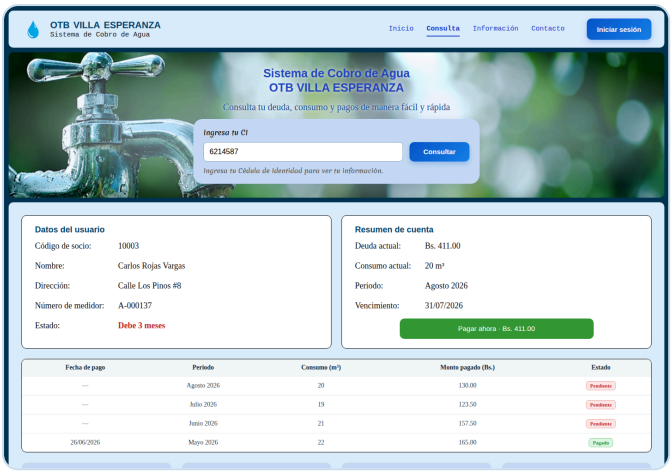
Eso solo queda bien en una pantalla del mismo tamaño que la del que lo diseñó. Un celular mide más o menos 400 píxeles de ancho, así que ese enlace se iba afuera de la pantalla. Además el `body` tenía `overflow: hidden` y no dejaba bajar con el scroll.

Lo cambié por **Flexbox** y **Grid**. Con eso uno le dice al navegador cuántas columnas quiere y él mismo acomoda los elementos según el espacio que haya. Los colores, las fuentes, las imágenes y el orden de las secciones son los mismos del diseño.

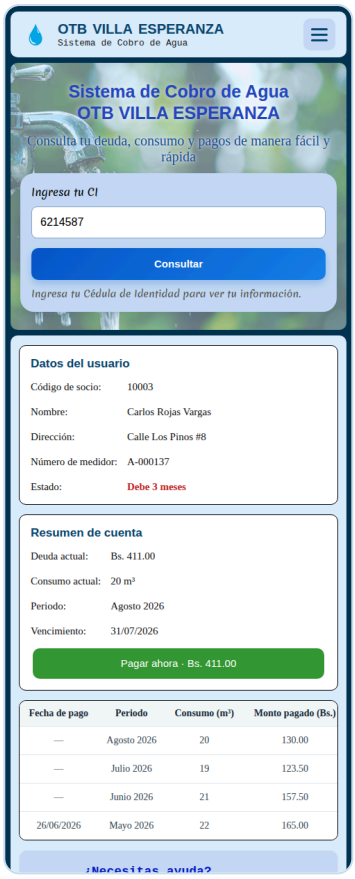
2.3 Lo hice responsive

Que una página sea responsive quiere decir que cambia según el tamaño de la pantalla. Usé cuatro tamaños:

Pantalla	Ancho	Qué cambia
Computadora grande	más de 1600 px	Se ve igual que el diseño original.
Laptop	hasta 1599 px	Las tarjetas de ayuda pasan abajo de los datos para que no se aprieten.
Tablet	hasta 1024 px	El menú se esconde y aparece el botón ☰. Las tarjetas van en 2 columnas.
Celular	hasta 700 px	Todo va en una columna, las letras son más chicas y los botones ocupan todo el ancho.



Consulta en computadora



La misma página en celular

2.4 Agregué el JavaScript

- **Buscador:** se escribe el carnet (CI) o el nombre y aparece la cuenta de esa persona.
- **Consulta:** muestra el código de socio, la dirección, el medidor, si está **Al día** o **Debe 3 meses**, cuánto debe y los pagos de los últimos 4 meses.
- **Pago con QR:** el vecino elige qué meses paga, se calcula el total y aparece el QR con una referencia para copiar. Después marca “Ya realicé el pago” y puede subir la foto del comprobante.
- **Información:** la tabla de tarifas se llena con las tarifas guardadas en el sistema.
- **Contacto:** el formulario revisa que no falten datos y guarda el mensaje para el administrador.
- **Panel:** inicio de sesión y permisos distintos según el tipo de usuario (lo explico en la sección 3).
- **Ventanas y avisos:** hice mis propias ventanas emergentes en lugar de usar el `alert()` del navegador.

2.5 Otras correcciones

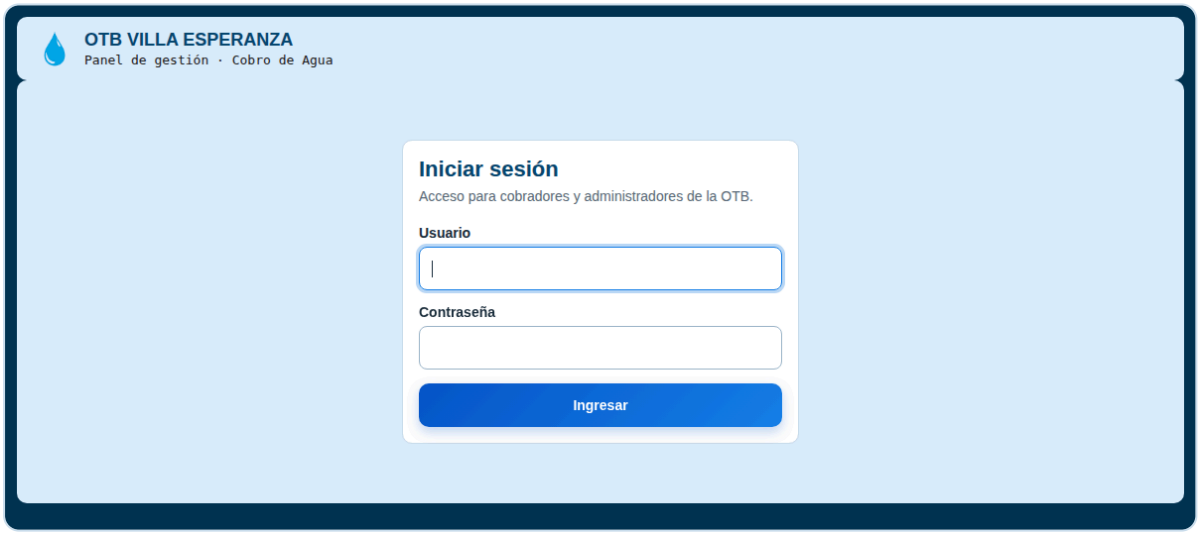
Antes	Ahora
Celula de Identidad	Cédula de Identidad
Jaun Pérez García	Juan Pérez García
12 m² (metros cuadrados)	12 m³, porque el agua se mide en metros cúbicos
No hay attención / hagamos un uno responsable	No hay atención / hagamos un uso responsable
El botón “Ir a Contacto” llevaba a Información	Ahora lleva a Contacto (`index3.html`)
El mensaje era un `<input>` de una línea	Lo cambié por un `<textarea>` de varias líneas
`lang="en"` y título “Document”	`lang="es"` y título “OTB VILLA ESPERANZA”
En Contacto el contenido estaba fuera del `<body>`	Lo moví adentro del `<body>`
El QR pesaba 274 KB	Lo reduje a 90 KB para que cargue más rápido

3 Usuarios y contraseñas

El panel (`panel.html`) es solo para el personal de la OTB. Se entra con el botón “Iniciar sesión” que está arriba a la derecha. Hay tres tipos de usuario. Cada uno puede hacer lo mismo que el anterior y algunas cosas más.

Usuario	Contraseña	Rol	Qué puede hacer
`cobrador`	`cobro123`	Cobrador	Ver el resumen, confirmar o rechazar pagos QR, registrar cobros en efectivo e imprimir el recibo, ver deudores y el historial de pagos, cambiar su contraseña.
`admin`	`admin123`	Administrador	Lo mismo que el cobrador, y además agregar o editar socios, cargar las lecturas de los medidores, cambiar tarifas, datos de contacto e imagen del QR, y leer los mensajes de Contacto.
`superadmin`	`super123`	Super Admin	Todo lo anterior, y además crear, editar, desactivar o eliminar usuarios, descargar y cargar respaldos, y volver a los datos de ejemplo.

Estas contraseñas son de prueba. Si el sistema se usa en serio hay que cambiarlas desde **Mi cuenta** dentro del panel.



Inicio de sesión del panel

Vecinos de prueba

Cargué 10 vecinos inventados para poder probar. Estos son algunos:

CI (carnet)	Nombre	Situación
4827163	Juan Pérez García	**Al día**
5930214	María Quispe Mamani	**Pago en verificación** (agosto)
6214587	Carlos Rojas Vargas	**Debe 3 meses**
3398712	Luis Mendoza Rocha	**Debe 5 meses**

También se puede buscar por nombre, por ejemplo *rosa condori*. No importa si se escribe con mayúsculas o sin tildes.

Dónde se guardan los datos

El proyecto no tiene servidor. Los datos se guardan en el mismo navegador usando `localStorage`, que es un espacio donde la página puede guardar información y la sigue teniendo aunque se cierre la pestaña.

Las contraseñas no se guardan como se escriben. Primero pasan por una función que las convierte en un código (a esto se le llama **hash**). Por ejemplo, `admin123` queda guardado como algo parecido a `9f3a61c0b27e44d5`. Cuando alguien inicia sesión, se convierte lo que escribió de la misma forma y se compara con lo guardado.

Limitaciones:
• Lo que registra un cobrador en su celular no aparece en otra computadora, porque cada navegador guarda sus propios datos. Para pasarlos de un equipo a otro está la opción **Respaldo** del Super Admin.
• Alguien que sepa programar puede ver el código y los datos. El inicio de sesión sirve para que no entre cualquiera, pero no es una seguridad fuerte. Para usarlo con muchos vecinos haría falta un servidor o algo como Firebase.

Cómo funciona un pago

1 El vecino busca su CI y aprieta **Pagar ahora**.

2 Le aparece el QR, el monto y la referencia (ej. `AGUA CI 6214587 JUN-2026 A AGO-2026`).

3 Paga desde la app de su banco y aprieta **Ya realicé el pago**.

4 El pago queda **En verificación**.

5 El cobrador revisa en el banco que el dinero llegó y lo **confirma**. Queda como **Pagado**.

4 Código HTML

El HTML es la estructura de la página: dice qué hay en ella (títulos, botones, tablas, imágenes). Cada elemento va dentro de etiquetas, por ejemplo `<p>...</p>` para un párrafo.

La parte de arriba del documento (`<head>`)

```HTML
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>OTB VILLA ESPERANZA</title>
    <link rel="stylesheet" href="css/estilos.css">
    <link rel="stylesheet" href="css/sistema.css">
</head>
```

- `lang="es"` indica que la página está en español.
- `charset="UTF-8"` permite usar tildes y la ñ.
- `viewport` es necesario para el responsive. Hace que el celular use el ancho real de su pantalla en vez de mostrar la página achicada.
- `<link>` conecta la página con los archivos CSS, que ahora están en la carpeta `css/` .

Nombre de cada página

```HTML
<body data-pagina="consulta">
```

Le agregué a cada página el atributo `data-pagina` con su nombre. El JavaScript lo lee para saber qué tiene que hacer (en consulta muestra la deuda, en contacto activa el formulario), y el CSS también lo usa para acomodar cada página de forma distinta.

El menú

```HTML
<div class="fofo">
    <a class="inicio" href="otb.html">Inicio</a>
    <a class="consulta" href="index1.html">Consulta</a>
    <a class="informacion" href="index2.html">Información</a>
    <a class="contacto" href="index3.html">Contacto</a>
    <button class="iniciar"><p class="sesion">Iniciar sesión</p></button>
</div>
```

`<a href="...">` es un enlace que lleva a otra página y `<button>` es un botón. `class` es el nombre que usan el CSS y el JavaScript para encontrar ese elemento. Dejé los nombres que ya tenía el diseño (`fofo` , `pepe` , `papa` ...) para no tener que cambiar todo.

La caja de búsqueda

```HTML
<div class="buscar">
    <input type="text" class="caja" placeholder="Ej: 1234567">
    <button class="pepe"><p class="consul">Consultar</p></button>
</div>
```

`<input>` es donde se escribe. `placeholder` es el texto gris de ejemplo que se borra al escribir. Desde JavaScript cambio ese texto por “Ej: 4827163 o Juan Pérez” y le digo al botón `.pepe` qué hacer cuando le hacen clic.

La tabla del historial

```HTML
<table>
    <thead>
        <tr><th>Fecha de pago</th><th>Periodo</th> ... </tr>
    </thead>
    <tbody>
        <tr><td>30/04/2026</td><td>Abril 2026</td> ... </tr>
    </tbody>
</table>
```

`<thead>` es la fila de títulos y `<tbody>` es donde van los datos. `<tr>` es una fila, `<th>` una celda de título y `<td>` una celda normal. Las filas que están en el HTML son de ejemplo. Cuando se busca a un vecino, el JavaScript las borra y pone las suyas.

Conectar el JavaScript

```HTML
    <script src="js/datos.js"></script>
    <script src="js/comun.js"></script>
    <script src="js/publico.js"></script>
</body>
```

Los puse al final del `<body>` para que primero se cargue la página. El orden importa: cada archivo usa cosas del anterior, así que primero van los datos, después las funciones comunes y al final las de cada página.

5 Código CSS

El CSS es lo que le da el aspecto a la página: colores, tamaños, posiciones y animaciones. Cada regla tiene un **selector** (a qué elemento se aplica) y, entre llaves, las **propiedades**.

Variables de colores

```css
:root {
    --fondo: rgb(3, 52, 80);
    --panel: #dbebfd;
    --titulo: rgb(9, 71, 112);
    --azul: #0756c9;
}
body { background-color: var(--fondo); }
```

> Guardé los colores del diseño en variables. Así en vez de repetir `rgb(3, 52, 80)` muchas veces escribo `var(--fondo)`, y si algún día hay que cambiar un color se cambia en un solo lugar.

Flexbox

```css
.buscar {
    display: flex;
    gap: 14px;
}
.buscar .caja { flex: 1; }
```

> `display: flex` pone los elementos uno al lado del otro y `gap` es el espacio entre ellos. Con `flex: 1` la caja de texto ocupa todo el espacio que sobra y el botón se queda de su tamaño.

Grid en la cabecera

```css
.arriba {
    display: grid;
    grid-template-columns: auto auto auto auto 1fr auto;
    align-items: center;
}
.arriba .otb  { grid-column: 2; }
.arriba .otb1 { grid-column: 3; }
.arriba .otb2 { grid-column: 4; }
.fofo         { grid-column: 6; }
```

> `display: grid` divide la cabecera en columnas: el logo, “OTB”, “VILLA”, “ESPERANZA”, un espacio que se estira (`1fr`) y el menú. Con `grid-column` indico en qué columna va cada cosa. Esto es lo que reemplazó a los `left: 1070px` del diseño original.

Media queries

```css
@media (max-width: 700px) {
    .medio h1 { font-size: clamp(20px, 5.4vw, 25px); }
    .buscar .pepe { flex: 1 1 100%; }
    .papa { grid-template-columns: minmax(0, 1fr); }
}
```

Lo que está dentro de `@media (max-width: 700px)` solo se aplica cuando la pantalla mide 700 píxeles o menos, o sea en celular. En este caso:

- El título usa `clamp(20px, 5.4vw, 25px)` : el tamaño cambia según el ancho de la pantalla (`vw`), pero nunca baja de 20 ni pasa de 25 píxeles.
- El botón Consultar ocupa todo el ancho y queda debajo de la caja.
- Los datos pasan de dos columnas a una.

Menú hamburguesa

```css
.otb-hamburguesa { display: none; }       /* en computadora no se ve */

@media (max-width: 1024px) {
    .otb-hamburguesa { display: inline-flex; }  /* en tablet y celular sí */
    .fofo { display: none; }                    /* el menú se esconde */
    .arriba.otb-menu-abierto .fofo { display: flex; } /* y se muestra al abrirlo */

    .otb-menu-abierto .otb-hamburguesa span:nth-child(1) {
        transform: translateY(8px) rotate(45deg);
    }
    .otb-menu-abierto .otb-hamburguesa span:nth-child(2) { opacity: 0; }
}
```

El botón ☰ son tres rayas (`span`). Cuando el menú está abierto, la primera y la tercera giran 45 grados y la del medio desaparece (`opacity: 0`), así el ícono se convierte en una X para cerrar. `nth-child(1)` se refiere al primer elemento.

Efecto al pasar el mouse

```css
.iniciar, .pepe {
    background: linear-gradient(135deg, var(--azul), var(--azul2));
    transition: transform 0.2s ease, box-shadow 0.2s ease;
}
.iniciar:hover  { transform: translateY(-2px); }
.iniciar:active { transform: scale(0.96); }
```

`linear-gradient` hace el degradado de un azul a otro. Con `:hover` el botón sube 2 píxeles cuando el mouse pasa por encima, y con `:active` se achica un poco al hacer clic. `transition` hace que el cambio dure 0.2 segundos para que no sea brusco.

Tabla que se desliza en el celular

```css
.tabla { overflow-x: auto; }
@media (max-width: 1024px) {
    .historial table { min-width: 560px; }
}
```

> La tabla del historial tiene cinco columnas y en un celular no entra. Le puse un ancho mínimo de 560 px y con `overflow-x: auto` se puede deslizar hacia los costados con el dedo.

Ocultar la barra del panel

```css
.p-usuario { display: flex; }
.p-usuario[hidden] { display: none; }
```

> La barra con “Ver sitio” y “Salir” no debe verse si nadie inició sesión. Tenía el atributo `hidden`, pero `display: flex` hacía que igual se mostrara. Con la segunda línea se oculta cuando tiene `hidden`.

6 Código JavaScript

El JavaScript es lo que hace que la página responda cuando uno hace clic, escribe o envía algo. Lo separé en cuatro archivos:

Archivo	Para qué sirve
`datos.js`	Los datos de ejemplo: 10 vecinos, las tarifas, la cuenta del QR y los 3 usuarios.
`comun.js`	Funciones que usan todas las páginas: guardar datos, iniciar sesión, buscar, ventanas, avisos y el menú ☰.
`publico.js`	Lo de las páginas de los vecinos: buscar, ver la deuda, pagar con QR, tarifas y contacto.
`panel.js`	Todo el panel de cobradores y administradores.

Listas y objetos (datos.js)

```javascript
var tarifas = [
    { desde: 0,  hasta: 10,   precio: 5.00 },
    { desde: 11, hasta: 20,   precio: 6.50 },
    { desde: 21, hasta: 30,   precio: 7.50 },
    { desde: 31, hasta: null, precio: 8.50 }
];
```

Los corchetes `[ ]` forman una lista (arreglo) y las llaves `{ }` un objeto, que guarda varios datos con nombre (`desde` , `hasta` , `precio`). `null` quiere decir que no tiene límite. Son las mismas tarifas que aparecen en la página de Información.

Calcular cuánto se paga

```javascript
function calcularMonto(m3) {
    for (var i = 0; i < tarifas.length; i++) {
        var t = tarifas[i];
        if (t.hasta === null || m3 <= t.hasta) {
            return Math.round(m3 * t.precio * 100) / 100;
        }
    }
    return 0;
}
```

La función recibe los m³ que se consumieron. Con el `for` revisa las tarifas una por una, y cuando encuentra el rango donde entra el consumo (`if`) multiplica los m³ por el precio. El `Math.round(... * 100) / 100` redondea a dos decimales.
Ejemplo: 12 m³ está en el rango de 11 a 20, entonces 12 × 6.50 = Bs. 78.00.

Guardar y cargar datos con localStorage (comun.js)

```javascript
var BD = {
    cargar: function () {
        var datos = JSON.parse(localStorage.getItem(CLAVE_BD));
        if (!datos) datos = this.datosIniciales();
        return datos;
    },
    guardar: function () {
        localStorage.setItem(CLAVE_BD, JSON.stringify(this._cache));
    }
};
```

`BD` es la base de datos del proyecto y tiene dos funciones, cargar y guardar.

- `localStorage.getItem` lee lo guardado y `setItem` lo guarda.
- localStorage solo guarda texto. Por eso uso `JSON.stringify` para pasar los datos a texto y `JSON.parse` para volver a convertirlos.
- La primera vez que se abre la página no hay nada guardado, así que se cargan los datos de `datos.js`.

Iniciar sesión y permisos

```javascript
var Sesion = {
    iniciar: function (usuario, clave) {
        var u = db.usuarios.find(function (x) { return x.usuario === usuario; });
        if (!u || u.hash !== hashClave(u.usuario, clave))
            return { ok: false, error: 'Usuario o contraseña incorrectos.' };
        sessionStorage.setItem(CLAVE_SESION, JSON.stringify(s));
        return { ok: true };
    },
    tieneRol: function (rolMinimo) {
        var orden = { cobrador: 1, admin: 2, superadmin: 3 };
        return orden[s.rol] >= orden[rolMinimo];
    }
};
```

- `find` busca al usuario en la lista.
- La contraseña escrita pasa por `hashClave` y se compara con la guardada. Si no son iguales, sale el error.
- Si está bien, la sesión se guarda en `sessionStorage`, que se borra cuando se cierra el navegador.
- `tieneRol` le da un número a cada rol (1, 2 y 3). La sección Usuarios pide nivel 3, así que el cobrador (nivel 1) no puede entrar ahí.

Buscar a un vecino por CI o por nombre

```javascript
function normalizar(texto) {
    return String(texto).toLowerCase()
        .normalize('NFD').replace(/[\u0300-\u036f]/g, '')
        .trim();
}
// "  Rosa CÓNDORI " -> "rosa condori"

function buscarSocios(texto) {
    var q = normalizar(texto);
    if (/^\d+$/.test(q)) {                        // ¿son solo números?
        return socios.filter(function (s) { return s.ci === q; });
    }
    var palabras = q.split(' ');
    return socios.filter(function (s) {
        return palabras.every(function (w) {
            return normalizar(s.nombre).indexOf(w) !== -1;
        });
    });
}
```

- `normalizar` pasa el texto a minúsculas y le quita las tildes y los espacios de más. Así "CÓNDORI" y "condori" se toman como iguales.
- `/^\d+$/` es una expresión regular que revisa si el texto tiene solo números. Si es así, busca por carnet.
- Si no, separa las palabras con `split` y se queda con los vecinos cuyo nombre tiene todas esas palabras (`every`).

Esperar a que cargue la página y responder a los clics

```javascript
document.addEventListener('DOMContentLoaded', function () {
    var pagina = document.body.getAttribute('data-pagina');
    prepararBuscador(pagina);
    if (pagina === 'consulta')    iniciarConsulta();
    if (pagina === 'informacion') mostrarTarifas();
    if (pagina === 'contacto')    iniciarContacto();
});

boton.onclick = buscar;
caja.addEventListener('keydown', function (e) {
    if (e.key === 'Enter') buscar();
});
```

- `addEventListener` se usa para ejecutar algo cuando pasa un evento. `DOMContentLoaded` es el momento en que la página terminó de cargar.
- Ahí se lee el `data-pagina` que puse en el HTML para saber qué preparar.
- Con `onclick` , al hacer clic en Consultar se ejecuta `buscar` . También funciona con la tecla Enter.

Mostrar los datos en la página

```javascript
var tbody = document.querySelector('.historial tbody');
var filas = cuenta.detalle.slice(-4).reverse();
tbody.innerHTML = filas.map(function (d) {
    return '<tr>' +
        '<td>' + nombrePeriodo(d.periodo) + '</td>' +
        '<td>' + d.m3 + '</td>' +
        '<td>' + bs(d.monto) + '</td>' +
    '</tr>';
}).join('');
```

- `querySelector` busca un elemento del HTML por su clase.
- `slice(-4)` toma los últimos 4 meses y `reverse()` los invierte para que el más reciente quede arriba.
- `map` arma una fila `<tr>` por cada mes y `join('')` las junta.
- `innerHTML` reemplaza lo que había en la tabla por esas filas.

Elegir los meses a pagar

```javascript
checks.forEach(function (ch) {
    ch.onchange = function () {
        var i = +ch.dataset.i;
        checks.forEach(function (otro) {
            var j = +otro.dataset.i;
            if (ch.checked && j < i)  otro.checked = true;
            if (!ch.checked && j > i) otro.checked = false;
        });
        actualizar();   // recalcula el total y la referencia
    };
});
```

Cada mes que se debe tiene una casilla. Hice que los meses se paguen en orden, empezando por el más antiguo:

- Si se marca julio, junio se marca solo.
- Si se desmarca junio, julio y agosto se desmarcan solos.

Así no se puede pagar agosto y dejar junio sin pagar. Al final `actualizar()` vuelve a calcular el total.

Achicar la foto del comprobante

```javascript
var escala = Math.min(1, maximo / Math.max(img.width, img.height));
var canvas = document.createElement('canvas');
canvas.width  = Math.round(img.width  * escala);
canvas.height = Math.round(img.height * escala);
canvas.getContext('2d').drawImage(img, 0, 0, canvas.width, canvas.height);
listo(canvas.toDataURL('image/jpeg', 0.7));
```

Las fotos del celular pesan mucho y localStorage tiene poco espacio (unos 5 MB). Por eso la foto se dibuja en un `canvas` más chico, de 900 px como máximo, y se guarda en JPG al 70% de calidad. Una foto de 3 MB queda en unos 100 KB.

Abrir y cerrar el menú ☰

```javascript
hamb.onclick = function () {
    var abierto = arriba.classList.toggle('otb-menu-abierto');
    hamb.setAttribute('aria-expanded', abierto ? 'true' : 'false');
};
```

`classList.toggle` pone la clase `otb-menu-abierto` si no está, y la quita si ya está. El CSS de la sección 5 se encarga de mostrar el menú y cambiar el ícono. `aria-expanded` le avisa a los lectores de pantalla (que usan las personas ciegas) si el menú está abierto o cerrado.

Evitar que se ejecute código escrito por el usuario

```javascript
function esc(texto) {
    return String(texto)
        .replace(/&/g, '&amp;').replace(/</g, '&lt;')
        .replace(/>/g, '&gt;').replace(/"/g, '&quot;');
}
```

Si alguien escribe algo como `<script>` en el buscador, la página podría tomarlo como código. La función `esc` cambia los símbolos `<` y `>` por otros equivalentes para que se muestren como texto y no se ejecuten.

Las pestañas del panel según el rol (panel.js)

```javascript
var SECCIONES = [
    { id: 'verificar', texto: 'Verificar pagos', rol: 'cobrador' },
    { id: 'socios',    texto: 'Socios',          rol: 'admin' },
    { id: 'usuarios',  texto: 'Usuarios',        rol: 'superadmin' },
    ...
];
var permitidas = SECCIONES.filter(function (sec) {
    return Sesion.tieneRol(sec.rol);
});
```

Esta lista tiene todas las pestañas del panel y el rol mínimo que se necesita para cada una. Con `filter` se muestran solo las que le corresponden al usuario: el cobrador ve 6, el admin 10 y el super admin 12.

7 Palabras que conviene conocer

Responsive	Página que se adapta al tamaño de la pantalla.
Etiqueta	Lo que va entre `< >` en HTML: `<p>` párrafo, `<a>` enlace, `<img>` imagen.
Clase (class)	Nombre que se le da a un elemento para darle estilo o encontrarlo con JavaScript.
Flexbox y Grid	Formas de ordenar elementos en filas, columnas o cuadrículas sin usar píxeles fijos.
Media query	Regla de CSS que se aplica solo en ciertos tamaños de pantalla.
Variable	Nombre que guarda un valor: un número, un texto o un color.
Función	Conjunto de instrucciones con nombre que se puede usar varias veces.
Evento	Algo que pasa en la página: un clic, una tecla, que termine de cargar.
localStorage	Espacio del navegador donde la página guarda datos que no se borran al cerrarla.
Hash	Código que se saca de una contraseña y no se puede volver atrás.
Rol	Tipo de usuario (cobrador, admin, super admin). Define qué puede hacer cada uno.
Modal	Ventana que aparece encima de la página, como la del pago con QR.

En resumen: partí de un diseño que solo se veía bien en una pantalla y no tenía funciones. Ordené los archivos, hice que se adapte a computadora, tablet y celular, programé la consulta de deudas y el pago con QR, y armé un panel con tres tipos de usuario para que la OTB maneje los cobros.